

Memoria de TFG/TFM: estructura y recomendaciones

Table of contents

1	1. Objetivo de este documento	2
2	2. Estructura recomendada de la memoria	2
3	3. Introducción	3
4	4. Contexto y estado del arte	3
5	5. Objetivos y alcance	4
5.1	5.1. Objetivo general	4
5.2	5.2. Objetivos específicos	4
5.3	5.3. Alcance y limitaciones	4
6	6. Metodología	4
7	7. Análisis y requisitos	5
8	8. Diseño y arquitectura	5
9	9. Implementación	6
10	10. Evaluación / experimentación	7
11	11. Conclusiones y trabajo futuro	7
12	12. Referencias	8
13	13. Anexos	8
14	14. Recomendaciones prácticas	9

15 15. Recursos complementarios recomendados	9
15.1 15.1. Libro sobre TFG/TFM en la UGR	9
15.2 15.2. Plantilla de TFG de JJ Merelo (ETSIIT)	10

1 1. Objetivo de este documento

Este documento describe la **estructura recomendada** para la memoria de tu TFG/TFM y algunas pautas de calidad.

No sustituye a la normativa oficial de la ETSIIT o de la UGR (que siempre tiene prioridad), pero:

- Te da una estructura base razonable para la mayoría de proyectos de Ingeniería Informática.
- Te ayuda a no dejar la memoria para el último momento: la idea es que se vaya **escribiendo en paralelo** al desarrollo.

2 2. Estructura recomendada de la memoria

La estructura concreta puede variar según el proyecto, pero una plantilla razonable es:

1. **Introducción**
2. **Contexto y estado del arte**
3. **Objetivos y alcance del proyecto**
4. **Metodología**
5. **Análisis y requisitos**
6. **Diseño (y arquitectura)**
7. **Implementación**
8. **Evaluación / Experimentación**
9. **Conclusiones y trabajo futuro**
10. **Referencias**
11. **Anexos**

En los apartados siguientes se detalla qué se espera en cada sección.

3 3. Introducción

La introducción debe responder, en pocas páginas, a:

- ¿Cuál es el **problema** que aborda el proyecto?
- ¿Por qué es **relevante**?
- ¿Cuál es la **situación previa** (muy resumida)?
- ¿Qué se pretende conseguir con este TFG/TFM?

Elementos recomendados:

- Breve contexto (sin todavía entrar a fondo en el estado del arte).
 - Objetivo general del trabajo (una frase clara).
 - Estructura de la memoria (describir brevemente qué contiene cada capítulo).
-

4 4. Contexto y estado del arte

Aquí debes:

- Analizar el contexto técnico y/o social del problema.
- Revisar trabajos relacionados, herramientas, librerías, sistemas similares, etc.

Recomendaciones:

- Agrupa los trabajos por temática (no hagas una lista sin hilo conductor).
- Compara y discute:
 - En qué se parecen a tu propuesta.
 - En qué se diferencian.
 - Qué hueco deja libre el estado del arte que tu TFG/TFM intenta cubrir.

En proyectos basados en software:

- Puedes analizar soluciones comerciales, proyectos open source, frameworks, etc.

En proyectos más “experimentales”:

- Puedes centrarse en técnicas, algoritmos, metodologías previas.
-

5 5. Objetivos y alcance

Aunque hayas mencionado los objetivos en la introducción, aquí se formalizan:

5.1 5.1. Objetivo general

- Una frase que describa el propósito principal del TFG/TFM.

5.2 5.2. Objetivos específicos

- Lista numerada de objetivos concretos (3–7 es un rango razonable).

Ejemplo genérico:

- Diseñar e implementar una aplicación que...
- Evaluar el rendimiento / precisión / usabilidad de...
- Integrar X y Y para demostrar que...

5.3 5.3. Alcance y limitaciones

- ¿Qué **sí** se va a hacer?
 - ¿Qué **no** entra en el alcance (límites del TFG/TFM)?
 - Justificar las decisiones (tiempo, complejidad, etc.).
-

6 6. Metodología

En este capítulo se explica **cómo** se ha trabajado:

- Metodologías de desarrollo (por ejemplo, enfoque ágil).
- Metodologías de investigación o experimentación (si aplica).
- En proyectos con personas usuarias:
 - Cómo se han realizado entrevistas, tests de usabilidad, encuestas, etc.

Recomendaciones:

- No basta con mencionar “Scrum” o “Kanban”: describe **cómo lo has adaptado** a tu TFG/TFM.
- Incluye:

- Cómo has organizado los sprints.
- Qué artefactos has usado (backlog, Project, issues).
- Cómo se han planificado y revisado los hitos.

Si tu proyecto utiliza técnicas específicas (p.ej. DCU, experimentos controlados, etc.), descríbelas aquí o en subapartados.

7 7. Análisis y requisitos

En esta sección se pasa del problema general a requisitos concretos.

Contenido típico:

- Perfiles de usuario y casos de uso (cuando tiene sentido).
- Historias de usuario (si se usa este enfoque).
- Requisitos funcionales y no funcionales:
 - Funcionales: qué debe hacer el sistema.
 - No funcionales: rendimiento, seguridad, usabilidad, escalabilidad, etc.

Recomendaciones:

- No te limites a una lista plana: organiza por áreas funcionales.
 - Asegúrate de que los requisitos estén relacionados con los objetivos del proyecto.
-

8 8. Diseño y arquitectura

Aquí se describe **cómo se ha diseñado la solución**, sin entrar todavía en detalles de código.

Secciones típicas:

1. Arquitectura de alto nivel

- Componentes principales.
- Cómo se comunican.
- Tecnologías elegidas (a alto nivel) y justificación.

2. Modelo de datos

- Esquemas de base de datos.

- Diagramas de clases / entidades.

3. Diseño de la interacción y la interfaz (si aplica)

- Flujos de usuario.
- Wireframes, prototipos.
- Decisiones de diseño importantes (por ejemplo, por accesibilidad).

4. Diseño de API o servicios (si aplica)

- Endpoints principales.
- Formato de datos (JSON, etc.).

Objetivo: que alguien con conocimientos técnicos pueda entender la forma general de tu solución sin mirar el código.

9 9. Implementación

En este capítulo cuentas **cómo se ha materializado el diseño en código**.

Secciones típicas:

- Descripción de los módulos más importantes:
 - Qué hacen.
 - Cómo se relacionan.
- Tecnologías concretas utilizadas:
 - Lenguajes, frameworks, librerías clave.
- Decisiones técnicas relevantes:
 - P.ej., por qué se eligió un framework frente a otro.
- Problemas encontrados y cómo se han resuelto.

No es necesario mostrar todo el código, pero sí:

- Fragmentos representativos.
 - Ejemplos de uso.
 - Referencias al repositorio en GitHub cuando sea relevante.
-

10 10. Evaluación / experimentación

Este capítulo responde a:

¿Qué tan bien funciona lo que has hecho?

Según el tipo de proyecto:

- **Aplicación de usuario final:**
 - Pruebas de usabilidad.
 - Pruebas de rendimiento, tiempos de respuesta, etc.
- **Proyecto de ciencia de datos / IA:**
 - Diseño de experimentos.
 - Conjuntos de datos utilizados.
 - Métricas y resultados (precisión, recall, F1, etc.).
- **Proyecto de infraestructura / sistemas:**
 - Pruebas de carga, mediciones de latencia, disponibilidad, etc.

Recomendaciones:

- Describe la metodología de evaluación:
 - Cómo se han diseñado los tests.
 - Qué métricas se han elegido y por qué.
 - Presenta resultados de forma clara:
 - Tablas y gráficos.
 - Comentarios y análisis crítico (no solo “estos son los números”).
-

11 11. Conclusiones y trabajo futuro

En este capítulo cierras el proyecto:

- **Resumen de lo conseguido:**
 - Repasar objetivos y explicar en qué grado se han cumplido.
- **Principales aportaciones:**
 - Técnicas, de diseño, de integración, etc.

- **Limitaciones:**
 - Qué no se ha podido hacer y por qué.
 - **Líneas de trabajo futuro:**
 - Qué caminos razonables quedan abiertos:
 - * Mejora de la solución.
 - * Ampliación a otros contextos.
 - * Integración con sistemas externos, etc.
-

12 12. Referencias

Se recomienda:

- Usar un gestor de referencias (BibTeX, Zotero, etc.).
- Mantener un estilo homogéneo (APA, IEEE, etc., según la normativa del centro).

Asegúrate de:

- Citar correctamente todo trabajo previo que utilices.
 - No abusar de enlaces sin contexto: mejor referencias formales cuando sea posible.
-

13 13. Anexos

En los anexos puedes incluir:

- Diagramas detallados.
- Especificaciones técnicas extensas.
- Cuestionarios, guiones de entrevistas.
- Listados largos de código (si son imprescindibles).
- Manuales de usuario, etc.

La idea es que la memoria principal no quede saturada, pero la información necesaria para reproducir resultados o entender detalles esté disponible.

14 14. Recomendaciones prácticas

- **No dejes la memoria para el final:**
 - Ve escribiendo en paralelo.
 - Puedes usar issues/tareas específicas del tipo “Escribir sección X”.
- **Escribe pensando en alguien que no estuvo contigo** durante el TFG/TFM:
 - Que pueda entender qué has hecho, por qué y cómo.
- **Coherencia interna:**
 - Asegúrate de que objetivos, metodología, resultados y conclusiones encajan entre sí.

La memoria es parte esencial de la evaluación: no solo importa el producto final, sino también **cómo cuentas el proceso y los resultados**.

15 15. Recursos complementarios recomendados

Además de este handbook, es muy recomendable que consultes otros recursos ya existentes en la UGR y en la ETSIIT, especialmente para profundizar en la parte **formal** de la memoria:

15.1 15.1. Libro sobre TFG/TFM en la UGR

- Publicación disponible en la producción científica de la UGR:
<https://produccioncientifica.ugr.es/documentos/67e6f8553f806d20a93a1b2a>

Este libro ofrece:

- Una visión general de qué es un TFG/TFM en el contexto de la UGR.
- Recomendaciones sobre estructura, estilo y redacción académica.
- Pautas sobre evaluación y aspectos formales.

15.2 15.2. Plantilla de TFG de JJ Merelo (ETSIIT)

- Repositorio: <https://github.com/JJ/plantilla-TFG-ETSIIT>

Es especialmente útil si:

- Quieres escribir la memoria en **LaTeX**.
- Prefieres partir de una plantilla ya adaptada a la ETSIIT.
- Te interesa ver un ejemplo de automatización del proceso de compilación.

Puedes usar:

- Este documento como guía sobre **qué** debe contener la memoria y cómo organizarla.
- El libro de la UGR y la plantilla de JJ como referencia para el **formato**, el estilo y buenas prácticas de redacción académica.